

CV Sorting System: Ultimate Developer Onboarding Guide

Welcome to the **CV Sorting System**. If you are a new developer taking over this project, this document is your complete roadmap. It contains everything you need to understand the system's architecture, database structure, security model, and codebase map.

1. Project Summary & Core Functionality

The **CV Sorting System** is an AI-powered recruitment dashboard designed to automate the screening of resumes. Instead of HR manually reading hundreds of CVs, they create a "Task" and upload a Job Description (JD).

The Pipeline:

- Creation:** A user creates a Task (e.g., "Software Engineer") and sets an AI Prompt.
- Sourcing:** Resumes are pulled either manually or via automated RPA (Robotic Process Automation) bots scraping external portals (like BDJobs).
- AI Screening:** The backend passes the CVs and JD to Large Language Models (OpenAI/Gemini/Local).
- Scoring:** The AI returns a `match_percentage` and detailed analysis for each candidate.
- Dashboarding:** HR logs into this system, views the ranked candidates, exports them to Excel, or emails the "Concern Person" (Hiring Manager) directly.

2. System Architecture Diagram

This diagram explains how the Frontend, Backend APIs, RPA Bots, and external AI services interact.

```
graph TD
    %% Entities
    Client[Browser UI / React Sidebar]
    API_Gateway[PHP Backend /api/]
    DB[(MySQL Database)]
    RPABot[Python/Node RPA Scraper Bot]
    BDJobs[BDJobs Portal]
    LLM[External AI: Gemini / OpenAI]

    %% Connections
    Client <-->|AJAX Polling & Fetch| API_Gateway
    API_Gateway <-->|Read/Write| DB

    %% RPA Flow
    RPABot -->|Polls Config & Limits| API_Gateway
    RPABot -->|Scrapes CVs| BDJobs
    RPABot -->|Uploads PDF & Data| API_Gateway

    %% AI Flow
    API_Gateway -->|Sends CV Text & Prompts| LLM
    LLM -->|Returns Match Score & JSON| API_Gateway
```

3. Database Entity-Relationship Diagram (ERD)

The system relies on a relational MySQL database. Below are the core tables and how they link together.

```
erDiagram
    USERS ||--o{ JOB_LIST : "creates"
    EMPLOYEES ||--|{ USERS : "maps to (employee_id)"
    JOB_LIST ||--o{ CANDIDATES : "has many"
    JOB_LIST ||--|{ PROMPTS : "uses"

    USERS {
        string username PK "Employee ID"
        string password
        string role "root, super-admin, admin, user"
        json permissions "Granular RBAC flags"
        string status "active/blocked"
    }

    JOB_LIST {
        string jd_id PK "Unique Job Hash"
        string task_no "e.g., TASK123"
        string created_by FK "Refers to Users"
        string status "created, pending, active, completed"
        string allowed_viewers "Comma separated user IDs"
        datetime created_at
    }

    CANDIDATES {
        int id PK
        string jd_id FK "Refers to Job_List"
        string candidate_name
        float match_percentage "AI Score"
        text ai_response "Detailed breakdown"
        boolean is_shortlisted
    }

    PROMPTS {
        int id PK
        string jd_id FK
        text prompt_text "The instructions for the AI"
    }
```

4. Authentication & Security Model (RBAC)

This system uses a strict **Role-Based Access Control (RBAC)** architecture evaluated on the backend for every request.

```
flowchart LR
    Root["Root Admin<br>ID: 097727"] -->|Bypasses all checks| SuperAdmin
    SuperAdmin["Super-Admin"] -->|Can manage| Admin
    Admin["Admin"] -->|Can manage| User["Standard User"]

    subgraph Granular_Permissions ["Granular Permissions (JSON)"]
        direction TB
        P1["manage_users"]
        P2["view_all_task"]
        P3["delete_task"]
    end
    end

    User -. -> |Requires explicit| Granular_Permissions
```

- **Backend Enforcement:** The function `has_permission($key)` in `config/auth.php` is the supreme authority. Even if a user hacks the frontend to show a hidden button, the PHP API will reject the POST request if they lack the JSON permission flag.
- **Hierarchy:** Lower-tier users can **never** view or edit the permissions of higher-tier users (`api/user_api.php` enforces this).

5. Feature & Modal Mapping

If you need to edit a specific UI element, here is exactly where to look:

Feature / UI Element	Frontend Code Location	Backend API Endpoint
		N/A (State driven by PHP Session)
React Navigation Sidebar	<code>includes/sidebar_modern.php</code>	

User Listing & Grid	js/script.js -> renderUserList()	api/user_api.php
User Permission Toggles	includes/modals.php (#permissionsModal)	api/user_api.php?action=update_permissions
Task Grid / My Tasks	js/script.js -> applyJobFilters()	api/get_jobs.php
Create New Task Modal	includes/modals.php (#addTaskModal)	api/insert_job.php
RPA / Bot Config	includes/modals.php (#rpaConfigModal)	api/rpa_config_api.php
Candidate DB / Excel Export	view/dashboard.php	api/export_candidates.php

6. Comprehensive API Dictionary (/api/)

The backend is modular. **Every API file checks the session authentication state at line 1.**

6.1 Authentication & Real-Time Sync

- **auth_check_api.php**: The frontend polls this every 3 seconds to keep React permissions synced.
- **forgot_password_api.php**: Handles password resets and token emails.
- **user_api.php**: Master controller for CRUD operations on users, inline role editing, and granular permissions.

6.2 Task & Job Flow

- **insert_job.php**: Validates Daily/Monthly caps (task_limit_api.php), generates JD tracking IDs, and writes to Job_List.
- **get_jobs.php**: Fetches the main dashboard list. It dynamically handles the "5-minute edit lifecycle" locking mechanism.
- **job_api.php**: Router for Deleting tasks, or explicitly updating allowed_viewers (Task Sharing).
- **update_task_status.php**: Pushes tasks linearly through states (pending -> active -> completed).

6.3 Candidate Data & Display

- **get_candidates.php**: The main engine for the candidate dashboard grid filtering and sorting.
- **view_cv.php / view_jd.php**: **Secure File Servers**. Never link to a PDF directly. These APIs serve the raw file bytes ensuring only authenticated users can view resumes.
- **export_candidates.php**: Dumps candidate metadata into a downloadable CSV/Excel file.

6.4 RPA Bot Hooks (Machine-to-Machine)

- **rpa_config_api.php**: Bots ping this to get their active sleep timers and batch chunk limits.
- **insert_candidate.php**: Bots POST scraped CV data and AI match scores here.
- **update_cv_count.php**: Pinged constantly by bots to update the frontend UI numbers live as it scrapes BDJobs.

6.5 Mail & Notifications

- **send_filtered_mail.php**: Dispatches an email to the Hiring Manager ("Concern Person") containing a secure link to view specific candidates.
 - **send_task_mail.php**: Automated background email sent to the task creator indicating AI screening is complete.
-

7. Exhaustive File-Level Codebase Map

7.1 Root Directory (/)

Essential Core Files

- **login.php & check_user.php**: Entry authentication gateway.
- **index.php & tasks.php**: The main Dashboards. index.php is the overview, tasks.php is the core data grid.
- **schema.sql**: The master database architecture blueprint.

● Unnecessary Files (Safe to Delete)

- ***.bak, *_test.php, test_*.php**: All test files can be wiped.
- **debug_*.php, check_*.php, fix_*.php**: Temporary console/fix scripts used during initial development.

7.2 Configuration (/config/)

- **db.php**: The \$conn MySQLi instance.
- **auth.php**: The heart of RBAC (has_permission()).
- **mail_sender.php**: PHPMailer wrapper for SMTP.

7.3 Includes & Modals (/includes/)

- `sidebar_modern.php`: React UI navigation. Needs to stay in sync with `auth_check_api.php`.
- `modals.php`: **CRITICAL FILE**. 90% of the HTML forms in the system exist hidden inside this file to keep `tasks.php` clean.

7.4 Views (/view/)

- `dashboard.php`: The actual Candidate View you see when clicking into a Task.
- (Legacy files like `dashboard_modern_old.php` can be deleted).

7.5 JavaScript (/js/)

- `script.js`: The monolithic ~5,000 line frontend controller. It holds `window.allJobsData`, renders tables, handles modal toggling, and executes `applyJobFilters()` for real-time search typing.
 - (Delete `script_test.js`).
-

Final Advice for New Developers

1. **Never trust the frontend**. Always enforce permission checks inside the `/api/` files.
2. If the React Sidebar isn't updating correctly, check the `permissionsLoaded` Event Listener inside `includes/sidebar_modern.php`.
3. The RPA bots rely heavily on exact API JSON responses. Be very careful modifying the output structures of `rpa_config_api.php` or `update_cv_count.php`.